Graph Coloring with CSR: Cerebras Approach

Siddharth Banakar

Contents

1 The Example Graph **3**

2 Building the CSR Representation **3**

3 CPU Sequential Coloring (Bucket Greedy) **3**
3.1 Algorithm Overview . . . 3
3.2 Setup . . . 4
3.3 Iteration-by-Iteration Walkthrough . . . 4
3.4 Final CPU Result . . . 5
3.5 Why It Works . . . 5

4 Cerebras WSE Speculative Parallel Coloring **5**
4.1 Algorithm Overview . . . 5
4.2 Setup . . . 6
4.3 Round 1 . . . 6
4.4 Round 2 . . . 7
4.5 Final Cerebras Result . . . 7

5 Deep Dive: How Wavelet Exchange Works **8**
5.1 The Problem . . . 8
5.2 Three Phases of Communication . . . 8
5.3 Why This Is Fast . . . 8

6 Why Conflicts Never Cascade: Deterministic Resolution **9**
6.1 The Rule: Higher ID Always Yields . . . 9
6.2 Why "Both Yield" or "Both Keep" Is Impossible . . . 9
6.3 What About the Next Round? . . . 9

7 Worst Case: All PEs in Conflict **9**
7.1 Setup . . . 9
7.2 Six Boundary Conflicts Detected . . . 10
7.3 Each PE Resolves Independently . . . 10
7.4 A Vertex Can Be in Multiple Conflicts . . . 10
7.5 Round 2 Recovery . . . 11

8 Barrier Synchronization Between Phases **11**
8.1 Why the Barrier After Wavelet Exchange Matters . . . 11
8.2 How Barriers Work on the WSE . . . 12
8.3 What Happens During the Wait . . . 12

9 Memory Efficiency: Fitting in 48 KB per PE **12**
9.1 The Wavelet Storage Problem . . . 12
9.2 Solution 1: Process Wavelets on Arrival . . . 13
9.3 Solution 2: Small Palettes by Design . . . 13
9.4 Solution 3: CSR Is the Tightest Packing . . . 13
9.5 48 KB Budget Breakdown . . . 14

10 Implementation: Checkerboard Routing and Software Relay **14**
10.1 The Routing Problem . . . 14
10.2 Checkerboard Routing . . . 14
10.3 Software Relay Buffers . . . 15
10.4 Wavelet Encoding . . . 15
10.5 Count-Based Completion Detection . . . 16

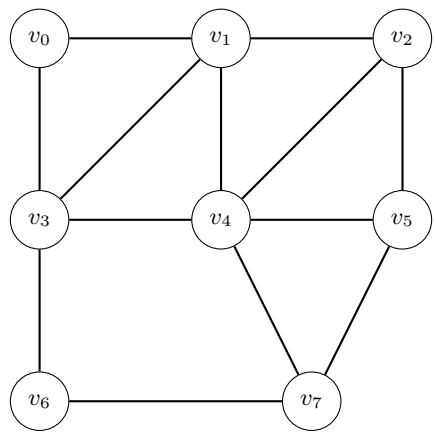
11 Implementation: Sync Barrier Protocol **16**
11.1 Phase 1: OR-Reduce (East → West) . . . 16
11.2 Phase 2: Broadcast (West → East) . . . 16
11.3 Two-Flag Gate: `maybe_proceed()` . . . 16

12 Implementation: Task Chaining and BSP Loop **17**
12.1 Task Chain Within One Round . . . 17
12.2 Termination . . . 17

13 Experimental Results **17**

1 The Example Graph

To keep things concrete, we will use the same small graph throughout this document: 8 vertices connected by 13 edges.



Edge list: $(0,1)$, $(0,3)$, $(1,2)$, $(1,3)$, $(1,4)$, $(2,4)$, $(2,5)$, $(3,4)$, $(3,6)$, $(4,5)$, $(4,7)$, $(5,7)$, $(6,7)$

Degrees: $\deg(v_0) = 2$, $\deg(v_1) = 4$, $\deg(v_2) = 3$, $\deg(v_3) = 4$, $\deg(v_4) = 5$, $\deg(v_5) = 3$, $\deg(v_6) = 2$, $\deg(v_7) = 3$

2 Building the CSR Representation

Before we can color anything, we need a way to store the graph in memory. CSR does this with just two flat arrays:

- `offsets[`n+1`]`: for vertex v, its neighbors live at `adj[offsets[`v`] .. offsets[`v+1`])`.
- `adj[`nnz`]`: flat array of all neighbor vertex IDs, packed contiguously.

Each undirected edge is stored as two directed arcs (both directions), so $|\texttt{adj}| = 2 \times 13 = 26$.

```
offsets[] = [0, 2, 6, 9, 13, 18, 21, 23, 26]
             v0 v1 v2  v3   v4   v5  v6  v7  end

adj[] = [1,3 | 0,2,3,4 | 1,4,5 | 0,1,4,6 | 1,2,3,5,7 | 2,4,7 | 3,7 | 4,5,6]
         v0     v1        v2      v3         v4          v5      v6    v7
```

Example lookup: neighbors of v_4.

$$\texttt{offsets}[4] = 13, \quad \texttt{offsets}[5] = 18 \quad \Longrightarrow \quad \texttt{adj}[13..18) = [1,\ 2,\ 3,\ 5,\ 7]$$

That is all it takes to find every neighbor of a vertex: two lookups into `offsets[]` and one contiguous walk through `adj[]`. No pointer chasing, no hash tables, just simple array arithmetic.

3 CPU Sequential Coloring (Bucket Greedy)

3.1 Algorithm Overview

Each vertex starts with a *color list*, a small, randomly chosen subset of the available colors. Here is how the CPU works through them, one vertex at a time:

1. Place vertices into **buckets** by color-list size (smallest first = most constrained).
2. Pick a vertex from the smallest non-empty bucket.
3. Assign it a color from its list.
4. **Propagate:** scan its CSR neighbors; remove the chosen color from each uncolored neighbor's list; move affected neighbors to smaller buckets.
5. Repeat until all vertices are processed.

Since we only color one vertex at a time and immediately tell its neighbors, **conflicts simply cannot happen**.

3.2 Setup

Let us say each vertex draws a random color list from the palette $\{C_0, C_1, C_2, C_3, C_4\}$:

Vertex	Color List	Vertex	Color List
v_0	$[C_0, C_2]$	v_4	$[C_0, C_1, C_3]$
v_1	$[C_1, C_2, C_3]$	v_5	$[C_0, C_2]$
v_2	$[C_0, C_1]$	v_6	$[C_1, C_3]$
v_3	$[C_0, C_2, C_3]$	v_7	$[C_0, C_1, C_2]$

Initial buckets:

- Bucket[2]: v_0, v_2, v_5, v_6 (2 colors each, most constrained)
- Bucket[3]: v_1, v_3, v_4, v_7 (3 colors each)

3.3 Iteration-by-Iteration Walkthrough

Iteration 1—Pick v_0 from Bucket[2].

- v_0 picks C_0 from its list $[C_0, C_2]$.
- Scan CSR: `offsets`$[0] = 0$, `offsets`$[1] = 2 \Rightarrow$ `adj`$[0..2) = [v_1, v_3]$.
- v_1: list $[C_1, C_2, C_3]$, C_0 not present $\rightarrow$ skip.
- v_3: list $[C_0, C_2, C_3]$, C_0 found $\rightarrow$ remove $\rightarrow [C_2, C_3]$.
- **Result:** $v_0 = C_0$. v_3 moves Bucket[3]$\rightarrow$Bucket[2].

Iteration 2—Pick v_2 from Bucket[2].

- v_2 picks C_1 from list $[C_0, C_1]$.
- CSR neighbors: `adj`$[6..9) = [v_1, v_4, v_5]$.
- v_1: $[C_1, C_2, C_3]$, C_1 found $\rightarrow$ remove $\rightarrow [C_2, C_3]$.
- v_4: $[C_0, C_1, C_3]$, C_1 found $\rightarrow$ remove $\rightarrow [C_0, C_3]$.
- v_5: $[C_0, C_2]$, C_1 not present $\rightarrow$ skip.
- **Result:** $v_2 = C_1$. v_1: Bucket[3]$\rightarrow$Bucket[2]. v_4: Bucket[3]$\rightarrow$Bucket[2].

Iteration 3—Pick v_5 from Bucket[2].

- v_5 picks C_2 from list $[C_0, C_2]$.
- CSR neighbors: `adj`$[18..21) = [v_2, v_4, v_7]$.
- v_2: already colored $\rightarrow$ skip.
- v_4: $[C_0, C_3]$, C_2 not present $\rightarrow$ skip.
- v_7: $[C_0, C_1, C_2]$, C_2 found $\rightarrow$ remove $\rightarrow [C_0, C_1]$.
- **Result:** $v_5 = C_2$. v_7: Bucket[3]$\rightarrow$Bucket[2].

Iteration 4—Pick v_6 from Bucket[2].

- v_6 picks C_1 from list $[C_1, C_3]$.
- CSR neighbors: `adj`$[21..23) = [v_3, v_7]$.
- v_3: $[C_2, C_3]$, C_1 not present → skip.
- v_7: $[C_0, C_1]$, C_1 found → remove → $[C_0]$.
- **Result:** $v_6 = C_1$. v_7: Bucket[2]→**Bucket[1]** (only 1 color left).

Iteration 5—Pick v_7 from Bucket[1].

- v_7 has only $[C_0]$, forced choice.
- CSR neighbors: `adj`$[23..26) = [v_4, v_5, v_6]$.
- v_4: $[C_0, C_3]$, C_0 found → remove → $[C_3]$.
- v_5, v_6: already colored → skip.
- **Result:** $v_7 = C_0$. v_4: Bucket[2]→Bucket[1].

Iteration 6—Pick v_4 from Bucket[1].

- v_4 has only $[C_3]$, forced choice.
- CSR neighbors: `adj`$[13..18) = [v_1, v_2, v_3, v_5, v_7]$.
- v_1: $[C_2, C_3]$, C_3 found → remove → $[C_2]$.
- v_3: $[C_2, C_3]$, C_3 found → remove → $[C_2]$.
- v_2, v_5, v_7: already colored → skip.
- **Result:** $v_4 = C_3$. v_1, v_3: Bucket[2]→Bucket[1].

Iterations 7–8—v_1 and v_3 (both forced).

- v_1 has only $[C_2]$ → $v_1 = C_2$.
- v_3 has only $[C_2]$ → $v_3 = C_2$.

3.4 Final CPU Result

v_0=C_0	v_1=C_2	v_2=C_1	v_3=C_2	v_4=C_3	v_5=C_2	v_6=C_1	v_7=C_0

8 sequential iterations, 4 colors used, 0 conflicts.

3.5 Why It Works

Notice how every iteration follows the same simple recipe:

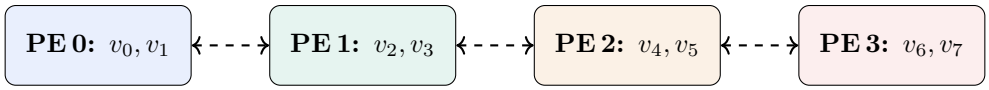
1. Look up `offsets[`v`]` and `offsets[`v+1`]` to find where this vertex's neighbors live in `adj[]`.
2. Walk through that contiguous slice to visit each neighbor.
3. If a neighbor has not been colored yet and the chosen color appears in its list, cross it off.

Because we propagate the color choice right away, no two neighbors can ever accidentally end up wearing the same color. The price we pay: **we can only color one vertex per step**, making the whole process strictly sequential.

4 Cerebras WSE Speculative Parallel Coloring

4.1 Algorithm Overview

The 8 vertices are **distributed across 4 Processing Elements (PEs)**, 2 vertices each:



Each PE holds its own slice of the CSR in local SRAM. The key difference from the CPU: all PEs work **at the same time**:

1. Every PE picks *tentative* colors for its uncolored vertices, without waiting for anyone else.
2. Each PE checks edges within its own partition for obvious clashes.
3. PEs then **exchange** their boundary colors with neighbors via tiny hardware messages called *wavelets*.
4. Each PE looks at the incoming messages and flags any conflicts on cross-PE edges.
5. Conflicts are resolved with a dead-simple rule: **the vertex with the higher ID backs off** and tries again next round.
6. Repeat until every vertex has a confirmed color.

4.2 Setup

We start with the same color lists as the CPU example, so we can compare apples to apples:

PE	Vertex	Color List
PE 0	v_0	$[C_0, C_2]$
PE 0	v_1	$[C_1, C_2, C_3]$
PE 1	v_2	$[C_0, C_1]$
PE 1	v_3	$[C_0, C_2, C_3]$
PE 2	v_4	$[C_0, C_1, C_3]$
PE 2	v_5	$[C_0, C_2]$
PE 3	v_6	$[C_1, C_3]$
PE 3	v_7	$[C_0, C_1, C_2]$

4.3 Round 1

Step 1: Speculate. All four PEs grab a tentative color at the same instant. Nobody waits for anyone else:

$$\begin{array}{lll} \text{PE 0:} & v_0 \to C_0 & v_1 \to C_1 \\ \text{PE 1:} & v_2 \to C_0 & v_3 \to C_0 \\ \text{PE 2:} & v_4 \to C_0 & v_5 \to C_0 \\ \text{PE 3:} & v_6 \to C_1 & v_7 \to C_0 \end{array}$$

Step 2: Local check (within each PE).

- PE 0: edge (v_0, v_1): $C_0 \neq C_1$, no conflict.
- PE 1: no edge between v_2, v_3, no conflict.
- PE 2: edge (v_4, v_5): $C_0 = C_0$, **LOCAL CONFLICT!** v_5 yields (ID $5 > 4$).
- PE 3: edge (v_6, v_7): $C_1 \neq C_0$, no conflict.

Step 3: Wavelet exchange. Each PE sends its boundary vertex colors to neighboring PEs:

- PE 0 sends $v_0{=}C_0,\ v_1{=}C_1$ to PE 1, PE 2.
- PE 1 sends $v_2{=}C_0,\ v_3{=}C_0$ to PE 0, PE 2.
- PE 2 sends $v_4{=}C_0,\ v_5{=}$uncolored to PE 0, PE 1, PE 3.
- PE 3 sends $v_6{=}C_1,\ v_7{=}C_0$ to PE 2.

Step 4: Detect boundary conflicts.

Edge	Colors	Match?	Resolution
(v_0, v_3)	$C_0 = C_0$	**Yes**	v_3 yields (ID $3 > 0$)
(v_1, v_3)	$C_1 \neq C_0$	No	N/A
(v_1, v_4)	$C_1 \neq C_0$	No	N/A
(v_2, v_4)	$C_0 = C_0$	**Yes**	v_4 yields (ID $4 > 2$)
(v_6, v_7)	$C_1 \neq C_0$	No	N/A

Round 1 result.

- **Confirmed (5):** $v_0{=}C_0,\ v_1{=}C_1,\ v_2{=}C_0,\ v_6{=}C_1,\ v_7{=}C_0$.
- **Yielded (3):** $v_3,\ v_4,\ v_5$ revert to uncolored for next round.

In one parallel round we colored 5 out of 8 vertices. The CPU needed 5 separate sequential steps to get this far.

4.4 Round 2

Step 1: Speculate (only uncolored vertices). Each vertex avoids colors already confirmed by its neighbors:

- v_3: neighbors $v_0{=}C_0, v_1{=}C_1 \Rightarrow$ avoid $\{C_0, C_1\} \Rightarrow$ picks C_2.
- v_4: neighbors $v_1{=}C_1, v_2{=}C_0, v_7{=}C_0 \Rightarrow$ avoid $\{C_0, C_1\} \Rightarrow$ picks C_3.
- v_5: neighbors $v_2{=}C_0, v_7{=}C_0 \Rightarrow$ avoid $\{C_0\} \Rightarrow$ picks C_2.

Step 2: Local check.

- PE 1: $v_3{=}C_2$ (only uncolored vertex on this PE) $\rightarrow$ no local conflict.
- PE 2: edge (v_4, v_5): $C_3 \neq C_2$, no conflict.

Step 3–4: Wavelet exchange + boundary check.

Edge	Colors	Match?
(v_3, v_4)	$C_2 \neq C_3$	No
(v_3, v_6)	$C_2 \neq C_1$	No
(v_5, v_7)	$C_2 \neq C_0$	No

No conflicts. All three vertices confirmed.

4.5 Final Cerebras Result

$v_0{=}C_0$	$v_1{=}C_1$	$v_2{=}C_0$	$v_3{=}C_2$	$v_4{=}C_3$	$v_5{=}C_2$	$v_6{=}C_1$	$v_7{=}C_0$

2 parallel rounds, 4 colors used, 3 conflicts resolved in round 1.

5 Deep Dive: How Wavelet Exchange Works

A natural question comes up: once every PE has picked its tentative colors, *how does it find out what colors the other PEs chose?* After all, each PE can only see its own vertices. The answer is **wavelet exchange**, a lightweight, hardware-routed messaging system built directly into the WSE fabric.

5.1 The Problem

Right after speculation, every PE is in the dark. It knows the colors it assigned to **its own** vertices, but has **absolutely no clue** what colors the other PEs chose. The local CSR tells it *who* the remote neighbors are, but says nothing about what color they just picked.

5.2 Three Phases of Communication

Phase 1—Speculate (purely local). Every PE picks tentative colors on its own. No messages fly yet.

Phase 2—Send wavelets. Now each PE walks through its local CSR looking for **boundary edges**, i.e., edges where one endpoint belongs to a different PE. For every such edge, it fires off a small message (a *wavelet*) through the hardware fabric:

```
wavelet = { vertex_id, color_assigned }
```

For example, PE 0 looks at v_0's neighbors in the CSR: $[v_1, v_3]$.

- v_1 is local (on PE 0) $\rightarrow$ no wavelet needed, check directly.
- v_3 is on PE 1 $\rightarrow$ **send wavelet** `(v0, C0)` to PE 1.

Similarly, PE 1 sends `(v3, C0)` to PE 0 (for edge $v_3 \leftrightarrow v_0$), and `(v3, C0)` to PE 2 (for edge $v_3 \leftrightarrow v_4$), etc.

On the WSE hardware, wavelets travel through the **2D mesh fabric**, a physical network connecting all 750,000+ PEs. Each wavelet is a small data packet routed along the grid. This takes only a few clock cycles, not a network round-trip.

Phase 3—Receive and detect. When PE 1 gets the wavelet `(v0, C0)`, it immediately checks: "My vertex v_3 is a neighbor of v_0, and I gave v_3 the color C_0 too. That is a **conflict!**" Since $3 > 0$, v_3 is the one that backs off and reverts to uncolored.

5.3 Why This Is Fast

Aspect	Detail
Wavelet size	Just a `(vertex_id, color)` pair, a few bytes
Transport	Hardware-routed on the wafer's 2D mesh, not software networking
Latency	Single-digit clock cycles between adjacent PEs
Parallelism	All PEs send and receive wavelets **simultaneously**

The whole speculate $\rightarrow$ exchange $\rightarrow$ detect $\rightarrow$ resolve cycle makes up a single "round," and every PE on the wafer runs through it at the same time.

6 Why Conflicts Never Cascade: Deterministic Resolution

6.1 The Rule: Higher ID Always Yields

The rule is almost trivially simple. Whenever two neighboring vertices end up with the same color and $i < j$:

- v_i (the lower ID) **keeps** its color.
- v_j (the higher ID) **gives up** and goes back to uncolored for the next round

The beauty of this rule is that both PEs can apply it **independently**. Neither one has to ask the other what to do.

6.2 Why "Both Yield" or "Both Keep" Is Impossible

Consider edge (v_0, v_3) with $C_0 = C_0$:

PE	Sees	Decides
PE 0	"My v_0 conflicts with v_3. Since $0 < 3$, **I win**."	Keeps $v_0 = C_0$
PE 1	"My v_3 conflicts with v_0. Since $3 > 0$, **I yield**."	Reverts v_3

This works because vertex IDs form a **total order**: for any two vertices, one ID is always strictly smaller than the other. There are no ties (every ID is unique), no coin flips, and no back-and-forth negotiation. Both PEs look at the same two numbers and always reach the same conclusion.

6.3 What About the Next Round?

After yielding, v_3 tries a **new** color in round 2 (say C_2). Could it clash with v_0=C_0 again? **No**, because v_0 is already locked in, and v_3 remembers from the wavelet it received that v_0 took C_0, so it steers clear.

But could v_3 run into a *different* neighbor that also just grabbed C_2? **Yes**, and if that happens, the same higher-ID-yields rule kicks in again. The point is that every round clears out all current conflicts, and the pool of uncolored vertices keeps shrinking. On average, about half the conflicting vertices survive each round, which is why the total number of rounds grows only as $O(\log n)$.

7 Worst Case: All PEs in Conflict

To really put the conflict-resolution rule through its paces, let us construct a nightmare scenario where **every single PE pair** ends up with a cross-PE conflict in Round 1.

7.1 Setup

Suppose every vertex speculatively picks one of just two colors:

PE Vertex	Vertex Tentative	Tentative	PE
PE 0	v_0	C_0	PE 2
v_4	C_0		
PE 0	v_1	C_1	PE 2
v_5	C_1		
PE 1	v_2	C_1	PE 3
v_6	C_0		
PE 1	v_3	C_0	PE 3
v_7	C_1		

7.2 Six Boundary Conflicts Detected

After wavelet exchange, every PE pair is involved in at least one conflict:

Edge	PEs	Colors	Conflict?	Resolution
(v_0, v_3)	PE 0 $\leftrightarrow$ PE 1	$C_0 = C_0$	**Yes**	v_3 yields $(3 > 0)$
(v_1, v_2)	PE 0 $\leftrightarrow$ PE 1	$C_1 = C_1$	**Yes**	v_2 yields $(2 > 1)$
(v_2, v_5)	PE 1 $\leftrightarrow$ PE 2	$C_1 = C_1$	**Yes**	v_5 yields $(5 > 2)$
(v_3, v_4)	PE 1 $\leftrightarrow$ PE 2	$C_0 = C_0$	**Yes**	v_4 yields $(4 > 3)$
(v_3, v_6)	PE 1 $\leftrightarrow$ PE 3	$C_0 = C_0$	**Yes**	v_6 yields $(6 > 3)$
(v_5, v_7)	PE 2 $\leftrightarrow$ PE 3	$C_1 = C_1$	**Yes**	v_7 yields $(7 > 5)$

7.3 Each PE Resolves Independently

Here is the key: no PE needs to talk to any other PE *about what to do*. Each one just looks at the vertex IDs and makes the call on its own:

PE	Vertex	Conflicts With	Decision
PE 0	v_0	v_3 $(0 < 3)$	**Keeps C_0**
PE 0	v_1	v_2 $(1 < 2)$	**Keeps C_1**
PE 1	v_2	v_1 $(2 > 1)$	**Yields**
PE 1	v_3	v_0 $(3 > 0)$	**Yields**
PE 2	v_4	v_3 $(4 > 3)$	**Yields**
PE 2	v_5	v_2 $(5 > 2)$	**Yields**
PE 3	v_6	v_3 $(6 > 3)$	**Yields**
PE 3	v_7	v_5 $(7 > 5)$	**Yields**

Result: Only v_0=C_0 and v_1=C_1 are confirmed; 6 vertices yield.

7.4 A Vertex Can Be in Multiple Conflicts

v_3 participates in **three** conflicting edges:

Edge	v_3's Role	Outcome
(v_0, v_3)	Higher ID $\rightarrow$ **yields**	v_3 loses
(v_3, v_4)	Lower ID $\rightarrow$ would keep	(irrelevant)
(v_3, v_6)	Lower ID $\rightarrow$ would keep	(irrelevant)

It only takes **one** losing conflict to knock a vertex out for the round. v_3 gives up because it lost to v_0. It does not matter that it would have beaten v_4 and v_6.

7.5 Round 2 Recovery

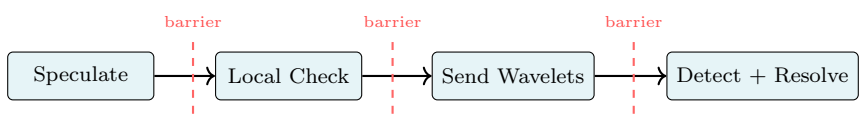
Now the 6 uncolored vertices are not starting from scratch; they remember the confirmed colors they learned from Round 1's wavelets:

Vertex	Confirmed Neighbors	Must Avoid	Picks
v_2	$v_1{=}C_1$	$\{C_1\}$	C_0
v_3	$v_0{=}C_0,\ v_1{=}C_1$	$\{C_0, C_1\}$	C_2
v_4	$v_1{=}C_1$	$\{C_1\}$	C_3
v_5	(none)	$\{\}$	C_2
v_6	(none)	$\{\}$	C_3
v_7	(none)	$\{\}$	C_0

No conflicts. All three vertices confirmed.

8 Barrier Synchronization Between Phases

You might wonder: do all PEs really stay in lockstep? Yes. Each round is split into **strict phases** separated by hardware barriers. No PE is allowed to jump ahead until every other PE has finished the current phase:



8.1 Why the Barrier After Wavelet Exchange Matters

Imagine PE 0 gets impatient and starts checking for conflicts **before** PE 2 has finished sending its wavelets. PE 0 would never see the message `''v4=C0''` and would wrongly conclude there is no conflict on edge (v_1, v_4). That would be a silent bug.

> **Every PE must finish sending AND receiving all wavelets before anyone starts checking for conflicts.**

8.2 How Barriers Work on the WSE

On a regular CPU cluster, barriers are painful; they mean network round-trips, OS scheduling overhead, and wasted time. On the Cerebras WSE, the story is completely different:

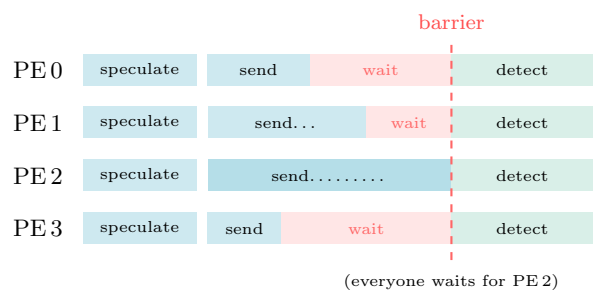
Mechanism	How It Works
Hardware barrier	The fabric has a built-in synchronization primitive: a single signal that propagates across the wafer in a few clock cycles
BSP model	Cerebras uses **Bulk Synchronous Parallel**: compute, communicate, sync, repeat. It is baked into the programming model (CSL/SDK)
Cost	A barrier across 750,000 PEs takes ∼tens of nanoseconds, a physical wavefront on silicon, not a software message

8.3 What Happens During the Wait

Think of it like a group of friends agreeing to wait at a checkpoint before moving on. A fast PE (say PE 3, which only has 2 boundary edges) finishes sending almost instantly and sits idle. A busy PE (say PE 2, with lots of boundary edges) takes a bit longer. During that gap:

- **The fast PEs just wait**, but the wait is measured in nanoseconds, not milliseconds, so it is practically free.
- **Nobody moves on** until the slowest PE catches up.

This is exactly why **good graph partitioning matters**: if one PE gets stuck with 10× more boundary edges than the rest, everyone else twiddles their thumbs waiting for it. A balanced partition keeps the slowest PE from becoming a bottleneck.



9 Memory Efficiency: Fitting in 48 KB per PE

Here is the hard constraint that shapes everything: each PE on the WSE-2 has only **48 KB of SRAM**. That is it: no cache, no DRAM, no disk to spill over to. The algorithm's code, all its data, and every incoming message must squeeze into that tiny budget. This is why the paper is titled "Picasso: **Memory-Efficient** Graph Coloring."

9.1 The Wavelet Storage Problem

Let us think about what PE 2 is dealing with. It holds v_4, which has 5 neighbors: v_1, v_2, v_3, v_5, v_7. After the wavelet exchange, PE 2 has received:

```
From PE 0:  "v1 = C1"
From PE 1:  "v2 = C0", "v3 = C0"
From PE 3:  "v7 = C0"
(v5 is local -- no wavelet needed)
```

That is 4 incoming wavelets for just one vertex's boundary neighbors. Now imagine what happens at real-world scale:

Graph Scale	Vertex Degree	Boundary Neighbors	Wavelets per PE
Toy (this example)	5	4	4
Real-world sparse	∼50	∼20	∼40
Dense molecular	∼500	∼200	∼400+

9.2 Solution 1: Process Wavelets on Arrival

The trick is to **never actually store the wavelets**. Instead, each PE has a *wavelet handler*, a tiny callback that runs the moment a wavelet shows up:

```
// Conceptual CSL handler on each PE
on_wavelet_received(sender_vertex, sender_color):
    for each local vertex v that neighbors sender_vertex:
        if v.tentative_color == sender_color:
            v.has_conflict = true    // just set a 1-bit flag
```

The wavelet gets consumed on the spot; it never sits in a buffer eating up precious memory. All we keep per vertex is a **1-bit conflict flag**, not the full history of every message.

9.3 Solution 2: Small Palettes by Design

Picasso also keeps palettes intentionally small. Each vertex gets roughly $\deg(v)/\log(n)$ colors, not the full rainbow. This keeps per-vertex storage very lean:

```
Per vertex storage:
  color           (1 int)         =  4 bytes
  has_conflict    (1 bit)         =  1 byte
  color_list      (~5 entries)    = 20 bytes
  CSR offset slice                = varies
```

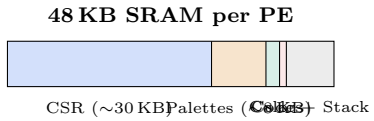
9.4 Solution 3: CSR Is the Tightest Packing

Representation	Memory for 100 local vertices, avg degree 50
Adjacency matrix	$100 \times N_{\text{total}}$ bits, **doesn't fit**
Adjacency list (pointers)	100 pointers + 5000 entries + pointer overhead, **borderline**
CSR	101 offsets + 5000 neighbor ints, **minimal, zero pointers**

CSR uses **zero pointers**: just two flat integer arrays laid out contiguously in memory. For sparse graph data, you simply cannot pack things any tighter than this.

9.5 48 KB Budget Breakdown

Putting it all together, the graph is sliced up so that each PE's portion fits neatly inside its 48 KB:



Component	Approximate Size
CSR offsets + adj	∼30 KB (for ∼500 vertices, degree ∼30)
Palette / color lists	∼8 KB
Color array	∼2 KB
Conflict flags	∼1 KB
Code + stack	∼7 KB
Total	**≤ 48 KB**

What if the graph is just too big? The answer is straightforward: **throw more PEs at it**. With up to 750,000 PEs on the WSE-2, even graphs with millions of vertices can be split into pieces small enough that every PE's share fits comfortably within its 48 KB.

10 Implementation: Checkerboard Routing and Software Relay

The high-level algorithm from Section 4 hides a critical engineering challenge: *how do wavelets actually reach their destination?* On the WSE, PEs are arranged in a 2D grid, and wavelet routing through the fabric requires careful configuration of **fabric colors** (hardware channels) to avoid deadlocks.

10.1 The Routing Problem

A wavelet from PE 0 to PE 3 must traverse PE 1 and PE 2. On the WSE, each PE has a **router** that can be configured per color: for a given color, incoming wavelets from one direction can be forwarded to another direction and/or delivered to the PE's local task (via RAMP).

The naïve approach—use a single color for eastbound traffic where every PE does `rx=WEST, tx=EAST,RAMP`—fails immediately. If PE 1 tries to *send* its own eastbound wavelet on the same color that pass-through traffic is using, the WSE hardware deadlocks: the PE cannot simultaneously inject a wavelet and receive a pass-through wavelet on the same color.

10.2 Checkerboard Routing

The solution is **checkerboard parity**: alternate send/receive colors based on PE position so that no PE ever sends and receives on the same color simultaneously.

We use **4 fabric colors** for east/west communication (colors 0–3):

- **Even-parity PEs** ($(row + col) \bmod 2 = 0$): send east on color 0, receive from east on color 2.

- **Odd-parity PEs** ($(row + col) \bmod 2 = 1$): send east on color 1, receive from east on color 3.

Each wavelet travels **exactly one hop** per fabric color switch. When PE 0 (even) sends a wavelet east on color 0, PE 1 (odd) receives it via the RAMP. PE 1's software then re-injects the wavelet eastward on color 1 (its own send color), and PE 2 (even) picks it up. This process repeats until the wavelet reaches its destination.

Route configurations per PE color (eastbound example):

Parity	Color	rx	tx
Even	C0 (send east)	RAMP	EAST
Even	C1 (recv east)	WEST	RAMP
Odd	C0 (recv east)	WEST	RAMP
Odd	C1 (send east)	RAMP	EAST

10.3 Software Relay Buffers

Since each wavelet only travels one hop at a time, intermediate PEs must **relay** wavelets that are not destined for them. Each PE maintains per-direction **circular relay queues**:

```
relay_east_q[max_relay]   // circular buffer for eastbound relay
relay_west_q[max_relay]   // circular buffer for westbound relay
```

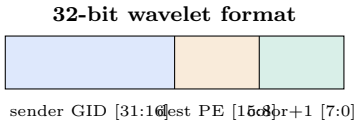
When a PE receives a wavelet, it inspects the packed `dest_pe` field:

- If `dest_pe == my_pe`: process immediately (update neighbor color table).
- If `dest_pe > my_pe`: push onto `relay_east_q` (forward east).
- If `dest_pe < my_pe`: push onto `relay_west_q` (forward west).

Drain rate. Relay wavelets are drained **one per task activation**. The send task alternates between sending the PE's own boundary wavelets and draining one relay entry. This "cooperative scheduling" between send and relay prevents output FIFO deadlock: between every send, receive data tasks can fire, keeping the pipeline fluid.

10.4 Wavelet Encoding

Each wavelet is a single 32-bit word, packed as follows:



- **Bits [31:16]**: Sender's global vertex ID (supports up to 65,535 vertices).
- **Bits [15:8]**: Destination PE index (supports up to 255 PEs in 1D).
- **Bits [7:0]**: Assigned color +1 (0 reserved, supports palette up to 255).

The destination PE is computed from the neighbor's global vertex ID using the partition chunk size: $\texttt{dest_pe} = \min(\lfloor \texttt{gid}/\texttt{chunk} \rfloor,\ P-1)$.

10.5 Count-Based Completion Detection

A key insight eliminates the need for "done" sentinel wavelets. The host **precomputes** the exact number of data wavelets each PE will receive per round:

$$\texttt{expected_total_recv}[p] = \sum_{q \neq p} \left| \{(u, v) \in \text{boundary}(q) : \texttt{gid_to_pe}(v) = p\} \right|$$

Each PE's receive handler decrements a counter `recv_remaining`. When it reaches zero, all data wavelets have arrived. Combined with the send-completion flag and relay-drained check, this precisely determines when the data exchange phase is complete—without any extra sentinel traffic.

11 Implementation: Sync Barrier Protocol

In 1D mode, a **software-implemented sync barrier** using 3 additional fabric colors (C8, C9, C10) ensures BSP semantics. The barrier has two phases:

11.1 Phase 1: OR-Reduce (East → West)

After all data wavelets are sent and received, the **eastmost PE** initiates a reduce chain:

1. PE_{N-1} (east edge) sends a sync wavelet westward containing its `has_uncolored` flag.
2. Each interior PE receives the sync wavelet from its east neighbor, OR-combines it with its own `has_uncolored` flag, and forwards the result westward.
3. PE_0 (west edge) receives the fully reduced value.

The reduce uses **alternating colors** (C8/C9) by PE parity to avoid same-color send/recv collision, identical to the checkerboard pattern for data.

11.2 Phase 2: Broadcast (West → East)

PE_0 sends the reduced `has_uncolored` flag eastward on color C10:

1. PE_0 sends broadcast wavelet east.
2. Each interior PE receives from the west, forwards east, and delivers to its local task.
3. PE_{N-1} receives and delivers locally.

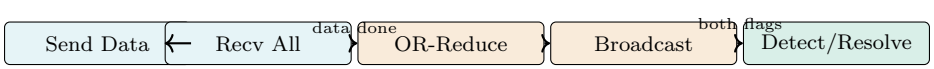
Color C10 uses `rx=WEST, tx={EAST, RAMP}` on interior PEs, which is safe because no PE injects its own wavelet on C10 (only PE_0 originates).

11.3 Two-Flag Gate: `maybe_proceed()`

Each PE tracks two flags:

- `local_data_done`: set when all expected data wavelets received.
- `sync_barrier_done`: set when broadcast received (or sent, for PE_0).

Only when **both** flags are true does the PE proceed to `detect_resolve`. This ensures no PE inspects boundary colors before every other PE has finished sending. If the reduced `has_uncolored` flag is zero (all vertices globally colored), the PE skips detection and exits immediately.



12 Implementation: Task Chaining and BSP Loop

The entire BSP round loop runs autonomously on the fabric with **zero host intervention** between rounds. The host only loads graph data, calls `start_coloring()`, waits for completion, and reads back the result.

12.1 Task Chain Within One Round

Each round follows this task chain:

1. `speculate`: Pick tentative colors for all uncolored vertices. Resolve intra-PE conflicts (higher GID yields). Activate `send_boundary`.
2. `send_boundary`: Sort boundary wavelets into per-direction buffers. Open receive window (`unblock_all_recv`). Activate `send_wavelet`.
3. `send_wavelet`: Send one wavelet per activation (own data or relay drain). Interleaves with `recv_*` data tasks automatically. Reaches phase 4 (done) and calls `check_completion`.
4. `check_completion`: When all own data sent, all expected wavelets received, and all relay buffers drained → triggers `on_data_complete()`.
5. `on_data_complete`: Initiates sync barrier (OR-reduce chain).
6. **Sync barrier**: `sync_reduce_recv` → `sync_bcast_recv` → `maybe_proceed()`.
7. `detect_resolve`: Check boundary conflicts, resolve (higher GID yields), reset state, activate `speculate` for a new round.

12.2 Termination

The sync barrier's OR-reduced flag tells every PE whether any vertex globally remains uncolored. If the flag is zero after at least one round, `maybe_proceed()` activates the `exit_task` directly, which stops the hardware cycle counter and unblocks the host command stream.

13 Experimental Results

We validated the implementation on the Cerebras SDK simulator across 8 test inputs derived from Pauli commutativity graphs, with PE counts from 4 to 16. Table 1 summarizes representative results.

Key observations:

- **All 41 eligible configurations passed** with correct colorings (zero conflicts, all vertices colored).
- **Non-power-of-2 PE counts** (5, 6, 7, 9, 10, 11, 13, 14, 15) all work correctly, validating the division-free routing.
- **Scaling:** Doubling PEs roughly halves cycle count for larger graphs (e.g., K16 at 4 PEs: 331K cycles → 16 PEs: 136K cycles, a 2.4× speedup).
- **Color quality:** Number of colors used matches or is close to the chromatic number. Small variations across PE counts arise from partition-dependent conflict patterns.

Table 1: Coloring results on the Cerebras simulator (1D layout, checkerboard routing).

Test Input	**Nodes**	**PEs**	**Colors**	**Cycles**
test1_all_commute	4	4	3	13,155
test2_triangle	4	4	2	7,570
test3_3pairs	6	4	3	43,164
test3_3pairs	6	6	3	22,673
test5_dense	8	4	3	39,638
test5_dense	8	8	3	22,180
test6_12nodes	12	4	5	219,199
test6_12nodes	12	8	5	143,840
test6_12nodes	12	12	6	112,710
test7_complete_K16	16	4	6	331,696
test7_complete_K16	16	8	7	191,841
test7_complete_K16	16	16	6	135,723
test8_structured	10	4	6	87,357
test8_structured	10	8	6	79,098
test8_structured	10	10	6	88,435